

LASER: Language Model Regression for Semi-Structured Workflow Resource and Runtime Estimation

Yuxuan Yin^{†1, 2}, Shengke Zhou¹, Yunjie Zhang¹, Ajay Mohindra¹, Boxun Xu² and Peng Li²

¹Google, ²University of California, Santa Barbara

Accurate prediction of resource consumption and runtime for cloud workflow jobs is critical for scheduling efficiency, yet remains challenging due to the semi-structured nature of job configurations—comprising shell commands, tool-specific parameters, dependency graphs, and hierarchical metadata. Traditional ML approaches require brittle feature engineering to flatten this rich information into fixed-size vectors, losing critical semantic context. We present LASER, a framework that fine-tunes LLMs on serialized workflow job configurations for multi-target resource and runtime regression. To address the challenges of numerical regression via generation, we introduce scientific notation output encoding for targets spanning multiple orders of magnitude, and constrained decoding with prefix filling to enforce output validity while reducing inference latency by over 30%. We further show that full-attention fine-tuning improves accuracy over sliding-window LLMs on long job contexts. Validated on large-scale chip design workloads, and GHARuntime, a new public benchmark derived from 580,000+ GitHub Actions runs across 27,000+ repositories, LASER outperforms human experts and SOTA tabular ML baselines, with clear model- and data-scaling behavior, establishing a new paradigm for LLM-based regression on semi-structured workflow data.

1. Introduction

Accurate prediction of runtime and resource consumption for cloud workflow jobs is fundamental to efficient scheduling, cost control, and infrastructure planning. Mispredictions lead to either *over-provisioning*—wasting expensive compute—or *under-provisioning*—causing job failures and costly re-runs (Liu et al., 2023; Stok, 2014). This challenge is particularly acute for modern workflow systems, which orchestrate complex pipelines of heterogeneous jobs across cloud and high-performance computing (HPC) infrastructures, where execution costs have skyrocketed as chip designs and scientific computations grow in complexity (Bavikadi et al., 2022; Ferreira da Silva et al., 2017). Workflows are typically structured as Directed Acyclic Graphs (DAGs) (Coleman et al., 2022), where each task—ranging from logic synthesis in large scale chip design simulations to sequence alignment in bioinformatics—carries a distinct configuration comprising tool invocations, command-line arguments, dependency specifications, and hardware constraints. Getting resource and runtime estimates right for such tasks is a prerequisite for backfill scheduling (Liu et al., 2025), makespan minimization (Bader et al., 2024a), and memory-efficient execution (Bader et al., 2024b; Lehmann et al., 2024).

Existing prediction approaches frame this as a tabular regression problem, requiring a feature-extraction function that maps complex job configurations into fixed-size vectors (Bader et al., 2024a; Liu et al., 2025). Statistical learning methods use handcrafted features derived from numerical metadata such as submission time, requested runtime, and

[†]Work done while the author was at Google

Correspondence to: shengkezhou@google.com, {y_yin, lip}@ucsb.edu

© 2026 Google. All rights reserved

processor counts (Feitelson et al., 2014), while deep learning methods extend this with temporal dependencies (Liu et al., 2025). However, this design forces practitioners to perform extensive and brittle feature engineering: command semantics (e.g., cross-compilation targets such as aarch64 versus x86_64), tool-specific parameters (e.g., python-version: 3.11 versus 3.9), and hierarchical structure (e.g., nested action configurations and inter-task dependencies) must all be hand-coded as scalar features (Huang, 2021; Zhu et al., 2024). Beyond being labor-intensive, such pipelines fail silently when new tools or commands appear—a common occurrence in rapidly evolving cloud services. Moreover, existing workflow-specific methods either rely on input-size-to-runtime linearity assumptions that do not hold universally in practice (Lehmann et al., 2024), or require costly online profiling runs before prediction can begin (Bader et al., 2024a). Critically, public workflow datasets such as WfCommons (Coleman et al., 2022) predominantly contain homogeneous task structures—e.g., the same binary applied to different input partitions—which further limits the scope of feature-based methods for predicting heterogeneous real-world workloads.

Large Language Models (LLMs) offer a fundamentally different approach. Having been pre-trained on vast corpora that include shell scripts, configurations, build systems, and domain-specific toolchains, LLMs can encode semantic signals from raw, semi-structured job configurations without any manual feature engineering. Recent work has demonstrated that LLMs can serve as effective regressors when fine-tuned appropriately through text-to-number generation (Akhaouri et al., 2025b; Song and Bahri, 2025; Song et al., 2024). Concurrently, Liu et al. (2025) showed that LLMs enhanced with retrieval-augmented generation (RAG) can leverage job script content to improve HPC runtime prediction—yet their approach requires per-query retrieval from a historical database, incurring significant inference latency, and cannot generalize to jobs unseen by the database. This opportunity for direct fine-tuning of LLMs on semi-structured workflow job configurations remains entirely unexplored.

et al., 2025b; Song and Bahri, 2025; Song et al., 2024). Concurrently, Liu et al. (2025) showed that

LLMs enhanced with retrieval-augmented generation (RAG) can leverage job script content to improve HPC runtime prediction—yet their approach requires per-query retrieval from a historical database, incurring significant inference latency, and cannot generalize to jobs unseen by the database. This opportunity for direct fine-tuning of LLMs on semi-structured workflow job configurations remains entirely unexplored.

We present LASER, a framework for Language model-bAsed regreSSion for sEmi-structured workflow job Resource and runtime estimation. LASER fine-tunes LLMs on serialized workflow job configurations for simultaneous prediction of wall-clock runtime, peak CPU, peak

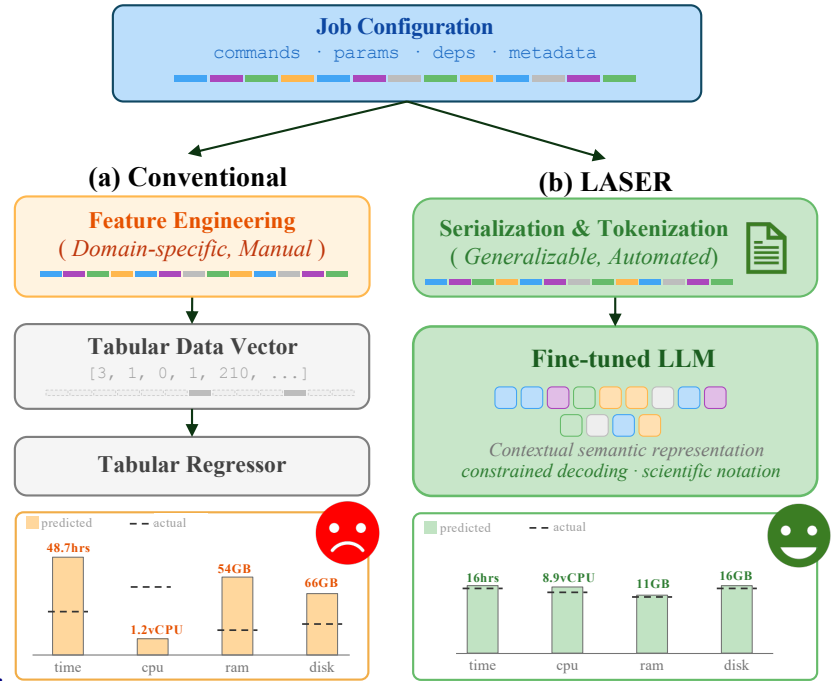


Figure 1 | Overview of conventional tabular prediction vs. LASER for workflow job resource and runtime estimation.

memory, and peak disk usage, treating the problem as a sequence-to-sequence generation task. Three techniques make this effective. First, we represent all numerical targets in scientific notation, providing a normalization-free, compact encoding for values spanning multiple orders of magnitude (e.g., memory from megabytes to terabytes) that avoids the training instability of linear regression heads (Song et al., 2024). Second, we introduce constrained decoding with deterministic prefix filling, which enforces structural output validity, eliminates hallucinations such as malformed JSON or fabricated keys, and reduces inference latency by over 30% by bypassing forward passes for deterministic structural tokens. Third, we demonstrate that full-attention fine-tuning substantially outperforms the default sliding-window attention of modern LLMs on long job configurations, where global context across distant command arguments and dependency specifications is critical for accurate prediction.

We validate LASER on two types of workflows. The first is a proprietary industrial chip design verification (ChipDV) workflow, comprising over 160,000 cloud jobs with rich semi-structured configurations of design verification workloads—domains where a minor change in a synthesis script can cause a tenfold increase in the runtime of the subsequent stage (Huang, 2021; Zhu et al., 2024). The second is GHARuntime, a new public benchmark we derive from the GHALogs corpus (Moriconi et al., 2025), containing over 1.3 million continuous integration and continuous deployment (CI/CD) workflow job execution records across 27,000+ GitHub repositories spanning 20 programming languages. Detailed backgrounds about these two datasets can be found in Appendix A.

In summary, we make the following contributions:

- We present the first framework for fine-tuning LLMs directly on semi-structured workflow job configurations for runtime and multi-resource prediction, eliminating the need for brittle feature engineering and outperforming both production heuristics and tabular ML baselines.
- We introduce scientific notation output encoding and constrained decoding with deterministic prefix filling, jointly improving prediction accuracy, output reliability, and inference efficiency by over 30%.
- We release **GHARuntime**, a large-scale public benchmark derived from the GHALogs dataset (Moriconi et al., 2025), comprising 1.3M+ CI/CD workflow job records with rich command-level metadata across 27K repositories, enabling reproducible research on workflow job runtime prediction.

2. Related Work

Language Model Regression. Treating regression as sequence generation has recently emerged as a viable alternative to classical regression heads. LLMs fine-tuned on (x, y) text pairs can outperform traditional regressors across diverse domains in Google Vizier’s blackbox optimization database (Song et al., 2024). Theoretical work shows that cross-entropy-trained decoders over tokenized numbers match pointwise regression heads in expectation (Song and Bahri, 2025). LLMs can also perform non-linear regression via in-context learning without parameter updates, rivaling Random Forest and Gradient Boosting (Vacareanu et al., 2024).

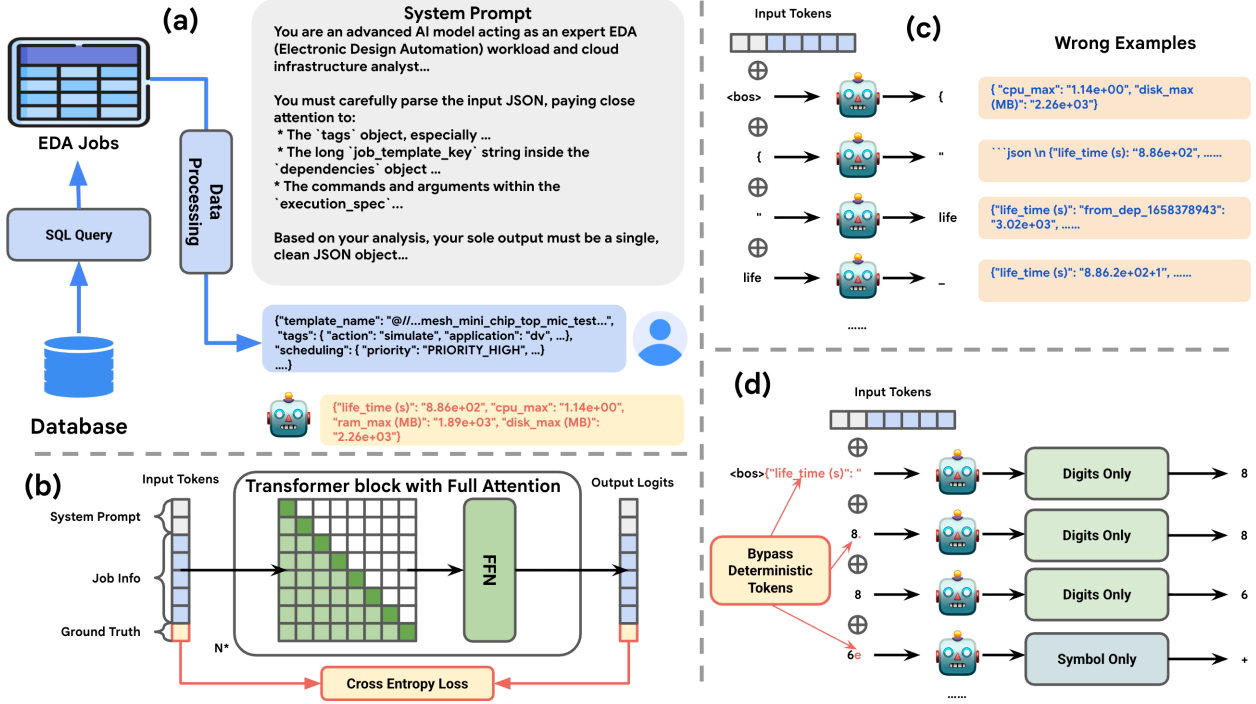


Figure 2 | Architecture of LASER for chip design verification workload prediction: (a) The data flow and conversational example. (b) Supervised fine-tuning LLM with full attention mechanism to minimize CE loss on ground truth tokens. (c) Vanilla LLM decoding methods and examples of wrong generations. (d) The proposed constrained decoding method.

Most relevant to our setting, text-to-text regression was applied to Google’s Borg cluster, achieving 100× lower MSE than tabular baselines on resource efficiency prediction (Akhaouri et al., 2025a), and was later extended to code-to-metric regression across 17 languages and GPU kernels using a unified Regression Language Model (Akhaouri et al., 2025b), with an encoder-decoder architecture. LASER differs from this line of work by aligning *decoder-only* LLMs on semi-structured workflow JSON with constrained numeric output generation and multi-target prediction.

Workflow and HPC Job Runtime Prediction. Classical HPC runtime prediction relies on numerical metadata from batch traces (Feitelson et al., 2014), with LightGBM emerging as the strongest tabular baseline (Chen et al., 2022). ORA augments LLMs with retrieval over job scripts and metadata, improving accuracy by over 40% versus metadata-only ML baselines (Liu et al., 2025). Unlike ORA, LASER uses supervised fine-tuning for single-pass inference without retrieval, generalizing to cold-start jobs from unseen repositories. For scientific workflow tasks, runtimes have been predicted via local profiling and Bayesian linear regression, assuming linear input-size-to-runtime relationships that rarely hold for EDA or CI/CD jobs (Bader et al., 2024a). Online memory sizing for Nextflow workflows has also been addressed by selecting among multiple ML models at the task-type level (Bader et al., 2024b; Lehmann et al., 2024)—a setting of repeated homogeneous tasks that is distinct from the heterogeneous job configurations in our datasets.

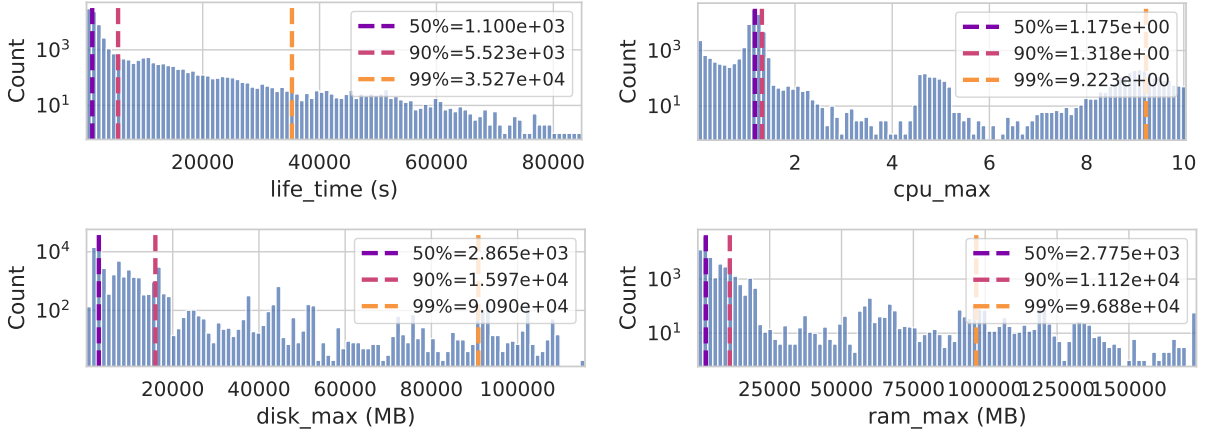


Figure 3 | Histograms illustrating the distribution of key resource metrics from dataset 1. All distributions are heavily right-skewed, indicating a long tail of resource-intensive jobs.

Workflow Datasets. WfCommons (Coleman et al., 2022) provides scientific workflow traces in standardized JSON, but most workflows consist of the same binary applied to different input partitions, offering limited semantic diversity for text-based methods. The Parallel Workload Archive (Feitelson et al., 2014) contains HPC batch traces in purely numerical SWF format with no script information. Our **GHARuntime** benchmark, derived from GHALogs (Moriconi et al., 2025), addresses both gaps with 1.3M CI/CD job records across 27K repositories featuring rich command-level metadata—to our knowledge the first public benchmark for workflow runtime prediction at this scale and semantic richness.

3. Methodology

We frame resource and runtime prediction as a sequence-to-sequence generation task: given a serialized job configuration as input, the model generates a structured output containing predictions for wall-clock runtime, peak CPU, peak memory, and peak disk usage. This formulation lets us leverage the full representational capacity of pre-trained LLMs and the training stability of Cross-Entropy loss $\mathcal{L}_{CE}$ to avoid the instability of linear regression heads over targets spanning multiple orders of magnitude. The key components of LASER: input serialization, numerical encoding, fine-tuning, and constrained decoding. These components are illustrated in Figure 2 and detailed in the following sections.

Input and Output Representation Let $\mathbf{x} \in \mathcal{X}$ denote a raw job configuration and $\mathbf{y} = [y^{(1)}, \dots, y^{(K)}] \in \mathbb{R}_{>0}^K$ the corresponding resource targets. We define a serialization function $\phi : \mathcal{X} \rightarrow \mathcal{V}^*$ that converts each configuration into a unified JSON token sequence, ordering fields by descending predictive importance so that the most informative content survives right-truncation to $L_{\max}$ tokens (ablated in Section 4.3). Targets are numerically encoded (Section 3.1) and serialized as a structured JSON output. We set $L_{\max}=2048$ for all experiments.

3.1. Scientific Notation Encoding

Resource metrics exhibit heavy right-skewed distributions spanning multiple orders of magnitude. Take our ChipDV dataset for an instance, `ram_max` ranges from the 50th percentile of 2.8×10^3 MB to the 99th percentile of 9.7×10^4 MB, while `life_time` spans from 1.1×10^3 s to 3.5×10^4 s in Figure 3. This long-tailed variation poses a fundamental challenge for regression heads: a linear or LogNormed MLP head fails to generalize across such ranges, and while output normalization partially mitigates this, it requires per-target, per-domain scaling that constitutes its own form of brittle feature engineering and balancing multi-objective optimization (Akhaouri et al., 2025b).

We leverage scientific notation to transform the regression problem into a next-token prediction problem. We encode each scalar target $y^{(k)} \in \mathbb{R}_{>0}$ as:

$$y^{(k)} = m^{(k)} \times 10^{e^{(k)}}, \quad m^{(k)} \in [1.00, 9.99], \quad e^{(k)} \in \mathbb{Z}, \quad (1)$$

$y^{(k)}$ is a fixed-length sequence of $L_{\text{fix}}=8$ tokens from $\mathcal{V}_{\text{num}} = \{0, \dots, 9, ., e, +, -\}$. The full prediction target $\mathbf{y}$ is then serialized as a JSON object with all K metrics encoded in this form, e.g., `{"life_time (s)": "8.86e+02", "cpu_max": "1.14e+00", ...}`. This gives two benefits: the mantissa is always bounded in $[1, 10)$, making per-target normalization unnecessary; and the fixed output length per metric simplifies constrained decoding (Section 3.3) and reduces generation cost.

Remark. Some LLM tokenizers split floating-point literals into variable-length, semantically opaque subword tokens, making it difficult to learn consistent numerical representations across orders of magnitude—a critical issue when targets range from megabytes to terabytes. We find that Qwen-3 (Yang et al., 2025) and Gemma-3 (Team et al., 2025) tokenizers encode each digit 0–9 as a single token, a property required for fixed-length scientific notation encoding. LLaMA-3, by contrast, merges digit sequences into multi-character tokens, violating this assumption. We therefore restrict our experiments to Qwen-3 and Gemma-3 architectures.

3.2. Supervised Fine-Tuning with Full Attention

Given a dataset of N job configurations and their corresponding resource targets $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$, we fine-tune a pre-trained LLM to generate the structured JSON output autoregressively. Each training example is formatted as a prompt-completion pair: the prompt contains the serialized job configuration $\tilde{\phi}(\mathbf{x}_i)$ wrapped in a standard instruction template, and the completion is the JSON-serialized target $\mathbf{y}_i$ with all numerical values encoded in scientific notation (Section 3.1). We minimize the cross entropy (CE) loss over completion tokens only:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \log p_{\theta}(w_t \mid w_{<t}), \quad (2)$$

where $\mathcal{T}$ indexes the completion tokens and p_{θ} is the LLM parameterized by θ .

Full-Attention Fine-Tuning. Modern long-context LLMs default to sliding-window attention to reduce computational cost, restricting each token’s receptive field to a local window. However, workflow job configurations not only have long contexts, varying hundreds to

Method	Test MAE (↓)				Pearson	Spearman
	life_time (min)	cpu_max (vCPU)	ram_max (GB)	disk_max (GB)	Avg. (↑)	Avg. (↑)
<i>ChipDV-Repetitive</i>						
Human Expert	—	0.311	54.90	66.70	—	—
Historical Max	—	0.295	16.20	21.70	—	—
LASER:						
Gemma-3-270M	69.29	0.301	11.90	7.14	0.469	0.509
Gemma-3-1B	51.30	0.183	3.29	1.39	0.834	0.773
Gemma-3-4B	47.02	0.168	2.61	1.08	0.829	0.821
Gemma-3-12B	38.02	0.128	1.89	0.80	0.913	0.869
Qwen-3-8B	41.07	0.130	1.66	0.76	0.919	0.858
<i>ChipDV-Unseen</i>						
Human Expert	—	0.330	53.60	59.08	—	—
LASER:						
Gemma-3-270M	37.73	0.478	3.45	3.75	0.502	0.438
Gemma-3-1B	27.91	0.165	2.30	1.30	0.767	0.701
Gemma-3-4B	26.25	0.169	1.56	0.95	0.832	0.745
Gemma-3-12B	23.76	0.145	1.40	1.14	0.858	0.799
Qwen-3-8B	24.52	0.136	1.41	0.94	0.856	0.804

Table 1 | Test results on industrial ChipDV workflow. — indicates the baseline does not produce a prediction for that target.

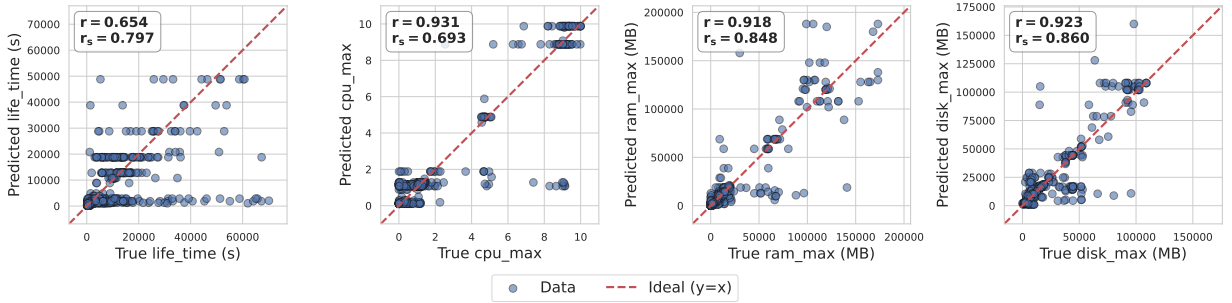


Figure 4 | True vs. predicted resource values from the Gemma-3-12B model on the test samples of dataset 1.

70k tokens of ChipDV workloads, but also encode predictive signals across distant fields, a compilation target specified early in the configuration may critically determine the runtime of a tool invoked much later. We find that replacing sliding-window attention with full attention during fine-tuning substantially improves prediction accuracy on long job contexts, and ablate this choice in Section 4.3.

Dataset	Test life_time (second)								
	GHARuntime-1k			GHARuntime-10k			GHARuntime-100k		
	MAE (↓)	Pearson (↑)	Spearman (↑)	MAE (↓)	Pearson (↑)	Spearman (↑)	MAE (↓)	Pearson (↑)	Spearman (↑)
XGBoost	645.20	0.02	0.09	485.03	0.21	0.58	487.01	0.21	0.60
LightGBM	812.13	−0.10	−0.06	464.73	0.23	0.56	468.43	0.28	0.52
TabPFN-v2.6	333.53	0.29	0.55	325.88	0.28	0.67	340.18	0.33	0.67
LASER:									
Qwen-3-0.6B	7324.92	−0.08	0.25	365.35	0.12	0.44	326.30	0.28	0.67
Qwen-3-8B	4001.35	0.27	0.50	308.25	0.24	0.60	312.20	0.33	0.72

Table 2 | GHARuntime test metrics of job lifetime.

3.3. Constrained Decoding with Prefix Filling

At inference time, the model must generate a well-formed JSON object whose values are valid scientific notation strings. Vanilla autoregressive decoding provides no such guarantee: the model may hallucinate malformed keys, emit invalid numerical formats, or produce structurally inconsistent outputs (see Figure 2(c)). We address this with constrained decoding with deterministic prefix filling.

The key observation is that the output structure is fully determined at inference time: the JSON keys, delimiters, and scientific notation scaffolding (e.g., {"life_time (s)": "<mantissa>e<exp>", ...}) are known constants, leaving only the mantissa digits and exponent as free variables. We therefore partition the output token sequence into *deterministic* tokens—structural tokens whose values are fixed by the output schema—and *generative* tokens—the $K \times L_{\text{fix}}$ numerical digits produced by the model. Deterministic tokens are filled directly without invoking a forward pass, while generative tokens are constrained to $\mathcal{V}_{\text{num}}$ via logit masking.

This yields two complementary benefits. First, output validity is enforced by construction: malformed JSON and invalid numerical formats are structurally impossible. Second, skipping forward passes for all deterministic tokens reduces inference latency by over 30%, a saving that grows with the number of predicted metrics K .

Finally, we observe that LLM may generate extreme values that are far beyond any training examples, due to our-of-range exponent digits. So we further clip model’s output to fit within the target value range of training examples.

4. Experimental Results

Experiments are conducted on (1) proprietary industrial ChipDV workflow, including multi-targets of job life time, peak usage of virtual CPU (vCPU)¹, memory, and disk; and (2) our derived GHARuntime workflow, a single target prediction task of job runtime. Due to industrial privacy constraints, ChipDV configuration details cannot be disclosed. LASER’s system prompt and GHARuntime examples are provided in Appendix B.

¹In our cloud platform, a vCPU is a virtualization abstraction representing a single hardware multithread on an underlying physical CPU processor.

We fine-tune Gemma-3 and Qwen-3 models with LoRA (Hu et al., 2022): we set LoRA rank as 8, LoRA alpha as 16, and use a dropout rate of 0.05. We train for 2 epochs with maximum sequence length 2048, learning rate 2×10^{-5} , and batch size 64. During testing generation, we set LASER LLM’s temperature to 0, leading greedy decoding to eliminate response randomness. We report mean absolute error (MAE), Pearson correlation (r), and Spearman correlation (r_s).

Gemma-3	1B	4B	12B
Vanilla	2.03	3.42	5.71
Constrained	1.38	2.18	3.99
Reduction	−32.4%	−36.1%	−30.1%

Table 3 | Inference time (s) for vanilla vs. constrained decoding.

We evaluate LASER against human experts, historical maximum records on ChipDV workloads; for GHARuntime, we compare our approach with competitive tabular regressors: XGBoost (Chen and Guestrin, 2016), Light GBM (Ke et al., 2017), and TabPFN-v2.6 (Hollmann et al., 2025)².

4.1. In-Distribution Performance

We evaluate LASER on two datasets sampled from the ChipDV workflow database with different schema: ChipDV-Repetitive (87,616 jobs, $\approx 64\%$ ones matching historical records), and a more challenging one ChipDV-Unseen (74,968 jobs, none of which can be found in the database). both dataset jobs are uniformly sampled from the same chip project within 10-day time window. We further split 80% data for training, 10% for validation, and 3,000 held-out examples for benchmarking, leading training and testing data to follow the same distribution.

As shown in Table 1, LASER significantly outperform human experts and historical check across both datasets. Figure 4 illustrates Gemma-3-12B predictions on ChipDV-Unseen, demonstrating LASER’s generalization across different metric value scales. We further observe the scaling law between model size and prediction accuracy: the two largest models, Gemma-3-12B and Qwen-3-8B, achieve the best performance across all targets.

4.2. Out of Distribution (OOD) Performance

A critical requirement for workflow resource predictors is generalization beyond the training distribution—either to future jobs under temporal drift, or to novel workflows from unseen codebases. We evaluate LASER on two OOD tasks:

- **Temporal drift:** fine-tuned on ChipDV-Repetitive, tested on future ChipDV jobs partitioned into 5-day windows (up to 26 days ahead);
- **Cross-repository generalization:** fine-tuned on a subset of GHARuntime repositories, tested on held-out repositories.

Temporal Drift. Table 4 reports MAE of Gemma-3-4B across six 5-day future windows.

²TabPFN-v2.6 checkpoint is released on March 24th 2026 along with no technical report or paper. We cite the journal version of TabPFN to compromise this situation.

Day Shift	Test MAE ($\downarrow$)			
	life_time (min)	cpu_max (vCPU)	ram_max (GB)	disk_max (GB)
0	47.02	0.168	2.61	1.08
1–5	62.19	0.154	3.06	1.79
6–10	76.86	0.157	3.37	1.73
11–15	30.27	0.161	2.92	1.52
16–21	33.36	0.173	2.71	1.52
22–26	44.65	0.146	2.50	1.78

Table 4 | Temporal OOD performance of LASER (Gemma-3-4B) on ChipDV, evaluated on non-overlapping future 5-day windows.

Val CE ($\downarrow$)	Test MAE ($\downarrow$)			
	life_time (min)	cpu_max (vCPU)	ram_max (GB)	disk_max (GB)
0.2873	61.00	0.253	6.19	3.24
0.2612	52.00	0.163	3.32	1.28
0.2515	46.83	0.143	2.23	0.96
0.2478	44.83	0.135	2.00	0.90
0.2463	41.07	0.130	1.66	0.76

Table 5 | Correlation between Validation Loss and Test MAE of Qwen-3-8B on ChipDV-Repetitive. Rows are ordered by training progress.

Performance on `cpu_max`, `ram_max`, and `disk_max` remains stable throughout the 26-day period, demonstrating strong resistance to temporal drift. `life_time` MAE is higher in the 6–10 day window but recovers in later windows, suggesting the model has learned generalizable resource consumption patterns rather than overfitting to the training period distribution.

Cross-Repository Generalization. Table 2 reports results on GHARuntime across three dataset scales (1k, 10k, 100k training jobs and 20% testing jobs). Two complementary scaling trends emerge. Beyond model-scaling improvement from Qwen-3-0.6B to Qwen-3-8B, LASER scales with data size as well: at 1k training samples both variants underperform ML baselines, but at 10k–100k they match or surpass TabPFN, v2.6—the strongest baseline. This data-scaling behavior highlights that LASER requires sufficient fine-tuning signal to adapt to workflow resource estimation task, while ML methods plateau early and fail to improve further with more data.

4.3. Ablation Study

Correlation between CE Loss and Prediction Accuracy. LASER minimizes cross-entropy (CE) loss during fine-tuning rather than a regression objective such as MAE. To verify CE loss is a reliable training signal, we evaluate Qwen-3-8B checkpoints at steps 500, 1000, 1500, 2000, and 2191 (epoch 2) on the ChipDV-Repetitive test set. As shown in Table 5, validation CE loss and test MAE across all four resource metrics show clear decreasing trend, with the final checkpoint achieving the lowest values on both objectives. This confirms that minimizing CE loss is an effective proxy for regression accuracy in workflow resource prediction.

Efficiency of Constrained Decoding. Beyond enforcing valid output format (Figure 2), constrained decoding also reduces inference latency. We measure total wall-clock generation time for standard vs. constrained decoding on ChipDV-Repetitive across Gemma-3-1B, 4B, and 12B. As shown in Table 3, constrained decoding achieves over 30% latency reduction on all three models, by skipping LLM forward passes for deterministic tokens and reducing the total tokens generated per example.

Full Attention vs. Sliding Window Attention. Gemma-3 models default to Sliding Window Attention (SWA: 512-token window for 270M/1B, 1024 for 4B/12B). We ablate this

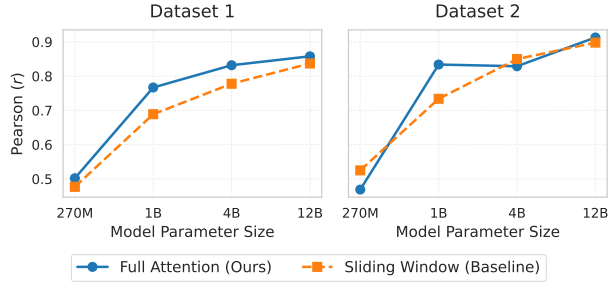


Figure 5 | Full attention vs. sliding window attention (SWA) across Gemma-3 model scales on ChipDV.

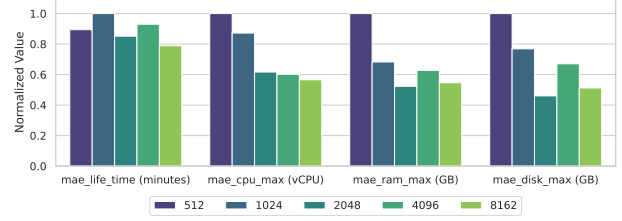


Figure 6 | Normalized MAE of Gemma-3-4B on ChipDV-Repetitive under varying maximum sequence lengths.

by fine-tuning both SWA and full-attention variants on ChipDV-Repetitive and comparing Pearson/Spearman correlation across model scales (Figure 5). Full attention consistently matches or outperforms SWA, with the largest gain at 1B. At 12B the gap narrows, but full attention still improves on both datasets. These results indicate that global context visibility is important for resource prediction of SWA LLMs.

Impact of Maximum Sequence Length We ablate maximum sequence length on ChipDV-Unseen using Gemma-3-4B, evaluating 512, 1024, 2048, 4096, and 8192 tokens, in Figure 6. Longer sequences improve accuracy by retaining more job context, but gains plateau beyond 2048—with 4096 and 8192 providing only marginal improvement. As 2048 also minimizes ram_max and disk_max MAE, we adopt it as the default.

5. Conclusion

We presented LASER, a framework for fine-tuning LLMs on semi-structured workflow job configurations for multi-target resource and runtime regression, eliminating the brittle feature engineering that limits traditional ML approaches. Three techniques make this effective: scientific notation encoding for targets spanning multiple orders of magnitude, constrained decoding with prefix filling that enforces output validity while reducing inference latency by over 30%, and full-attention fine-tuning for long job contexts. Validated on industrial chip design workloads and GHARuntime, a new public benchmark of 580,000+ GitHub Actions runs across 27,000 repositories, LASER consistently outperforms production heuristics and tabular ML baselines, with performance scaling predictably with model size and training data.

References

- Y. Akhauri, B. Lewandowski, C.-H. Lin, A. N. Reyes, G. C. Forbes, A. Wongpanich, B. Yang, M. S. Abdelfattah, S. Perel, and X. Song. Performance prediction for large systems via text-to-text regression. *arXiv preprint arXiv:2506.21718*, 2025a.
- Y. Akhauri, X. Song, A. Wongpanich, B. Lewandowski, and M. S. Abdelfattah. Regression language models for code. *arXiv preprint arXiv:2509.26476*, 2025b.
- J. Bader, F. Lehmann, L. Thamsen, U. Leser, and O. Kao. Lotaru: Locally predicting workflow task runtimes for resource management on heterogeneous infrastructures. *Future Generation Computer Systems*, 150:171–185, 2024a. doi: 10.1016/j.future.2023.08.016.
- J. Bader, F. Skalski, F. Lehmann, D. Scheinert, J. Will, L. Thamsen, and O. Kao. Sizey: Memory-efficient execution of scientific workflow tasks. In *Proceedings of the 2024 IEEE International Conference on Cluster Computing (CLUSTER '24)*, 2024b. doi: 10.1109/CLUSTER59578.2024.00039.
- S. Bavikadi, A. Dhavlle, A. Ganguly, A. Haridass, H. Hendy, C. Merkel, V. J. Reddi, P. R. Sutradhar, A. Joseph, and S. M. Pudukotai Dinakarrao. A survey on machine learning accelerators and evolutionary hardware platforms. *IEEE Design & Test*, 39(3):91–116, 2022. doi: 10.1109/MDAT.2022.3161126.
- I. Bouzenia and M. Pradel. Resource usage and optimization opportunities in workflows of github actions. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 279–290, 2024. doi: 10.1145/3597503.3623303.
- T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- Y. Chen, J. Yang, and S. Ren. Predicting HPC job completion time with supervised learning. In *Proceedings of the International Conference on High Performance Computing (HiPC)*, 2022.
- T. Coleman, H. Casanova, L. Pottier, M. Kaushik, E. Deelman, and R. Ferreira da Silva. WfCommons: A framework for enabling scientific workflow research and development. *Future Generation Computer Systems*, 128:16–27, 2022. doi: 10.1016/j.future.2021.09.043.
- D. G. Feitelson, D. Tsafir, and D. Krakov. Experience with using the parallel workloads archive. *Journal of Parallel and Distributed Computing*, 74(10):2967–2982, 2014. doi: 10.1016/j.jpdc.2014.06.013.
- R. Ferreira da Silva, R. Filgueira, I. Pietri, M. Jiang, R. Sakellariou, and E. Deelman. A survey on workflows and scientific workflow management systems in e-science environments. *International Journal of High Performance Computing Applications*, 31(1):1–3, 2017.
- N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025. doi: 10.1038/s41586-024-08328-6.

- E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- T.-W. Huang. Machine learning system-enabled GPU acceleration for EDA. In *Proceedings of the 2021 International Symposium on VLSI Design, Automation and Test (VLSI-DAT '21)*, pages 1–1, 2021. doi: 10.1109/VLSI-DAT52063.2021.9427323.
- G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NeurIPS '17)*, pages 3146–3154, 2017. URL <https://papers.nips.cc/paper/6907>.
- F. Lehmann, J. Bader, N. De Mecquenem, X. Wang, V. Bountris, F. Friederici, U. Leser, and L. Thamsen. Ponder: Online prediction of task memory requirements for scientific workflows. In *Proceedings of the 2024 IEEE 20th International Conference on e-Science (e-Science '24)*, 2024. doi: 10.1109/e-Science62913.2024.10678682.
- H. Liu, Y. Ma, X. Huang, L. Zhang, T. Jia, and Y. Li. ORA: Job runtime prediction for high-performance computing platforms using the online retrieval-augmented language model. In *Proceedings of the 2025 International Conference on Supercomputing (ICS '25)*, 2025. doi: 10.1145/3721145.3725757.
- M. Liu, L. Pan, and S. Liu. Cost optimization for cloud storage from user perspectives: Recent advances, taxonomy, and survey. *ACM Computing Surveys*, 55(13s), 2023. doi: 10.1145/3582883.
- F. Moriconi, T. Durieux, J.-R. Falleri, R. Troncy, and A. Francillon. GHALogs: Large-scale dataset of GitHub actions runs. In *Proceedings of the 22nd IEEE/ACM International Conference on Mining Software Repositories (MSR '25)*, pages 669–673, 2025. doi: 10.1109/MSR66628.2025.00104.
- X. Song and D. Bahri. Decoding-based regression. *Transactions on Machine Learning Research*, 2025. URL <https://openreview.net/forum?id=avUQ8jguxg>.
- X. Song, O. Li, C. Lee, B. Yang, D. Peng, S. Perel, and Y. Chen. OmniPred: Language models as universal regressors. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=t9c3pfrR1X>.
- L. Stok. The next 25 years in EDA: A cloudy future? *IEEE Design & Test of Computers*, 31(2): 40–46, 2014.
- G. Team, A. Kamath, J. Ferret, S. Pathak, N. Vieillard, R. Merhej, S. Perrin, T. Matejovicova, A. Ramé, M. Rivière, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.
- I.-L. Tseng. Pragmatic eda flow runtime prediction with machine learning and automatic outlier removal. In *2024 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*, pages 203–207, 2024. doi: 10.1109/APCCAS62602.2024.10808773.

- R. Vacareanu, V.-A. Negru, V. Suci, and M. Surdeanu. From words to numbers: Your large language model is secretly a capable regressor when given in-context examples. In *First Conference on Language Modeling (COLM)*, 2024. URL <https://openreview.net/forum?id=LzpaUxcNFK>.
- A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- L. Zhu, X. Ma, S. Hao, Y. Pan, and X. Guo. Elastic EDA: Auto-scaling cloud resources for EDA tasks via learning-based approaches. In *Proceedings of the 42nd IEEE International Conference on Computer Design (ICCD '24)*, pages 144–153, 2024. doi: 10.1109/ICCD63220.2024.00031.

A. Background on Chip Design and CI/CD Workflow Workloads

Modern cloud workflow jobs span domains from chip design to continuous integration and continuous deployment (CI/CD). Despite their surface differences, both share a defining structure: heterogeneous, semi-structured job configurations whose runtime and resource demands are difficult to predict from numerical metadata alone (Bavikadi et al., 2022; Liu et al., 2023).

Chip Design Workloads. Chip design workflows, formally electronic design automation (EDA) workflows, consist of a series of computational jobs such as logic synthesis, place-and-route, timing analysis, and physical verification (Stok, 2014). As semiconductor process technology advances, even a single sub-5-million-gate partition can require over 25 days of wall-clock time to complete a full EDA flow at the 3nm node (Tseng, 2024), making accurate runtime and resource prediction essential for project scheduling and cloud cost control (Zhu et al., 2024). Our industrial dataset focuses specifically on **Design Verification (DV)** workloads (Stok, 2014), where jobs exercise a chip’s functional correctness across combinatorial-scale simulation runs. The performance of DV jobs depends critically on four classes of factors. *Design Characteristics* encompass the size and complexity of the circuit under test, including the number of logic gates, memory blocks, and testbench stimulus files, all of which directly govern simulation time. *Tool Configuration* covers the specific EDA simulator, its version, and the multitude of settings and flags—such as coverage collection modes and compile-time optimizations—that can cause order-of-magnitude runtime differences between otherwise similar jobs (Huang, 2021). *Technology Node* refers to the target semiconductor process (e.g., 7nm, 5nm), which determines physical design rules and indirectly influences simulation model complexity. *Execution Environment* includes the underlying cloud VM type, storage tier, and license availability, all of which introduce significant runtime variability even for identical job configurations (Bavikadi et al., 2022). The interplay between these factors creates a high-dimensional, semi-structured feature space that resists conventional tabular encoding (Tseng, 2024; Zhu et al., 2024).

CI/CD Workloads. CI/CD workflow jobs, as captured in our **GHARuntime** benchmark, are triggered by repository events (e.g., push, pull_request, schedule) and execute on cloud-hosted virtual machines (Moriconi et al., 2025). GitHub Actions has emerged as the dominant CI/CD platform, with prior work showing that testing and building together consume around 90% of total CI/CD VM time (Bouzenia and Pradel, 2024). Each job is a sequence of heterogeneous steps—GitHub Actions with tool-specific parameters (e.g., actions/setup-python with python-version:3.9) and shell commands (e.g., cibuildwheel targeting aarch64)—whose runtimes span three orders of magnitude, from sub-minute utility checks to multi-hour cross-compilation builds.

Researchers recently observe that predicting a near-optimal timeout value for a CI/CD job requires reasoning about the job configuration itself and its repository history (Bouzenia and Pradel, 2024), a signal that purely numerical schedulers cannot capture. Both chip design and CI/CD workloads share the defining challenge: the most predictive signals reside in *textual, semi-structured* configuration fields that tabular feature engineering cannot faithfully encode (Huang, 2021; Liu et al., 2025).

B. Experimental Setup Details

B.1. GHARuntime System Prompt

The system prompt used for LASER on GHARuntime is as follows:

```
You are a precise CI/CD workflow job runtime forecaster.
INPUT: A JSON object describing a workflow job, including runner image,
action steps, and shell commands.
TASK: Predict the actual wall-clock runtime in seconds.
OUTPUT: Exactly one JSON object with no surrounding prose:
{"duration_sec": "<value>"}
NUMERIC FORMAT: Python f"{x:.2e}" notation.
- Exactly 2 fractional digits, e.g. 12300 → "1.23e+04"
- No leading + on mantissa, exponent always has sign.
```

B.2. GHARuntime Input Example

Below is a representative GHARuntime job record as consumed by LASER. The job belongs to the selinuxproject/refpolicy repository, triggered by a pull request on an ubuntu-20.04 runner, and has a ground-truth runtime of 206.0s (encoded as "2.06e+02").

```
{
  "repo": "selinuxproject/refpolicy",
  "event": "pull_request",
  "image": "ubuntu-20.04",
  "num_steps": 8, "num_shell": 6, "num_action": 2,
  "steps": [
    {"type": "action", "action": "actions/checkout",
      "with": {"fetch-depth": "1", "lfs": "false", ...}},
    {"type": "action", "action": "actions/setup-python",
      "with": {"python-version": "3.5", ...}},
    {"type": "shell", "code": "sudo apt-get install -qy bison flex ..."},
    {"type": "shell", "code": "make bare && make conf && make ..."},
    ...
  ]
}
```

```
{"duration_sec": "2.06e+02"}
```

B.3. Tabular Feature Representation

For tabular ML baselines (XGBoost, LightGBM, and TabPFN), the rich semi-structured job configuration is flattened into a fixed-size feature vector. This process discards hierarchical

structure and shell command semantics, retaining only hand-engineered scalar indicators. The feature set for the same example is shown below.

```
num_steps: 8, num_shell: 6, num_action: 2, total_code_len: 1079
is_ubuntu: 1, is_macos: 0, is_windows: 0
is_push: 0, is_pr: 1, is_schedule: 0
act_checkout: 1, act_setup-python: 1, act_setup-node: 0, ...
shell_make: 1, shell_pytest: 0, shell_npm test: 0, ...
has_python_ver: 0, has_node_ver: 0, has_aarch64: 0
```

This comparison illustrates the information loss inherent in tabular feature engineering: shell command content, action parameter values (e.g., `python-version: 3.5`), and step ordering are all discarded, while LASER consumes the full structured configuration directly.

C. GHARuntime: Dataset Construction

C.1. Source and Motivation

Public workflow datasets such as WfCommons (Coleman et al., 2022) predominantly contain homogeneous task structures in which the same binary is applied to different input partitions, providing limited semantic diversity for LLM-based prediction methods. To address this gap, we construct **GHARuntime**, a large-scale public benchmark for workflow job runtime prediction derived from the GHALogs corpus (Moriconi et al., 2025). GHALogs contains over 116,000 CI/CD workflows executed via GitHub Actions (GHA) collected from more than 25,000 public repositories across 20 programming languages, comprising approximately 513,000 workflow runs and 2.3 million individual steps. Unlike scientific workflow collections, GitHub Actions workflows exhibit *genuinely heterogeneous* job configurations: each job is a sequence of GitHub Actions and shell steps whose tool invocations, parameters, and runtime characteristics vary widely across repositories and programming ecosystems.

C.2. Extraction Pipeline

We process the GHALogs `runs.json` file (580,641 workflow run records) through the following steps:

Step 1: Filtering. We retain only workflow runs with `conclusion == "success"` to ensure that all runtime labels reflect complete, valid executions. Runs with missing or malformed `log_insights` fields are discarded.

Step 2: Job extraction. Each workflow run contains a `log_insights` list in which each entry corresponds to one CI/CD job (a parallel unit of execution assigned to a dedicated virtual machine). For each job, we sum the `duration_sec` field of all constituent steps to obtain the job-level wall-clock duration:

$$d_{\text{job}} = \sum_{s \in \text{steps}(j)} d_s. \quad (3)$$

Jobs with $d_{\text{job}} \leq 0$ or no steps are discarded.

Step 3: Serialization. Each job is serialized into a structured JSON object containing:

- **event:** the GitHub event that triggered the run (e.g., `push`, `pull_request`, `schedule`);
- **image:** the runner virtual machine image (e.g., `ubuntu-22.04`, `macos-12`, `windows-2022`);
- **steps:** an ordered list of step descriptors. For action-type steps, each descriptor includes the action name (e.g., `actions/setup-python`) and its with parameters (e.g., `python-version: "3.11"`). For shell-type steps, each descriptor includes the full shell command string (code field), truncated to 300 characters. Sensitive fields such as authentication tokens are filtered before serialization.

This serialization mirrors the input format used for the chip design simulation datasets, enabling the same LASER pipeline to be applied without modification.

Step 4: Train/test split. To ensure that test repositories are completely unseen during training—a stricter evaluation protocol than random splitting—we partition by *repository*: all jobs from 80% of repositories form the training set and all jobs from the remaining 20% form the test set. The split is deterministic (fixed random seed 42 after shuffling the sorted repository list).

C.3. Dataset Statistics

Table 6 | GHARuntime dataset statistics.

Statistic	Value
Total workflow runs (source)	580,641
Total workflow runs (after filtering failed runs)	440,109
Total jobs (after filtering)	1,322,504
Unique repositories	27,728
Unique workflow paths	33,310
Median job duration	100.3 s
90th-percentile duration	809.3 s
99th-percentile duration	3,865.7 s
Maximum duration	108,557.3 s
Std / Mean ratio (CoV)	2.80
Training jobs	~1.06M
Test jobs	~0.26M

The duration distribution is heavily right-skewed (coefficient of variation 2.80), spanning three orders of magnitude from sub-minute utility jobs (e.g., `twine check`, 12 s) to multi-hour cross-compilation builds (e.g., `aarch64 wheel` builds exceeding 4 hours). This range and skew closely mirror the distribution observed in the chip design simulation datasets

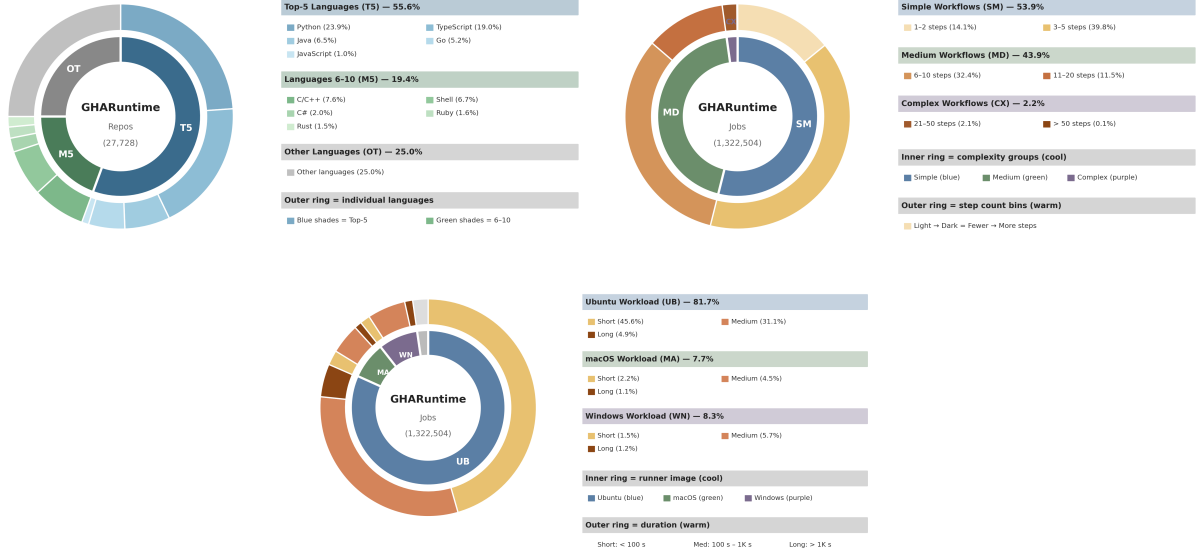


Figure 7 | Distribution statistics of GHARuntime job durations on training and test splits. The figure highlights the long-tail structure and split-level consistency.

(Figure 2 of the main paper), making GHARuntime a natural out-of-domain complement for evaluating LASER’s generalization.

Figure 7 summarizes the composition of GHARuntime across three dimensions. **By runner image**, Ubuntu dominates at 81.7% of all jobs, followed by Windows (8.3%) and macOS (7.7%), reflecting the prevalence of Linux-based CI/CD workflows on GitHub Actions. Within each platform, the duration breakdown reveals that short jobs (<100 s) constitute the majority of Ubuntu workloads (45.6%), while macOS and Windows jobs skew toward medium durations (100 s–1 Ks), consistent with their heavier use in build and compilation tasks. **By programming language**, the dataset spans 20 languages across 27,728 repositories, with Python (23.9%) and TypeScript (19.0%) as the two most represented. The top-5 languages account for 55.6% of repositories, while a long tail of 10+ additional languages contributes 25.0%, ensuring semantic diversity in workflow configurations. **By workflow complexity**, over half of all jobs (53.9%) involve simple workflows with 1–5 steps, while 43.9% fall in the medium range (6–20 steps). Only 2.2% of jobs exceed 20 steps, representing complex multi-stage pipelines such as matrix builds and cross-platform release workflows.

C.4. Why GHARuntime Enables LLM-Based Prediction

The key property distinguishing GHARuntime from prior workflow trace collections is the richness of *command-level text metadata* available at prediction time. Consider two jobs with identical structural metadata (same runner image, same number of steps): one runs `python -m tables.tests.test_all` (225 s) and another runs a `cibuildwheel` cross-compilation targeting `aarch64` (15,952 s)—a 70× difference invisible to any tabular feature.

LASER encodes the full shell command and action parameter strings, allowing it to capture such signals directly. In contrast, tabular ML baselines can only observe keyword indicators hand-engineered from the command text, necessarily losing precision on long-tail tool invocations.